

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The raw inertial signals were used for this experiment. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels. Thus, the input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset was obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

The experiment used 6250 training samples, 1102 validation samples, and 2947 testing samples. The sequence length was 128 with 9 features per timestep, classifying into 6 activity classes.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	6250
T	Time steps per sequence	128
F	Features/channels per time step	9

The model receives a three-dimensional tensor.

3. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models used the same input, output layer, optimizer, batch size and evaluation protocol.

4. Source Code

The complete implementation for this experiment is available at: <https://github.com/Jai-Abhishek/DL-Lab>

5. Preprocessing

The raw inertial sensor signals from the UCI Human Activity Recognition dataset were loaded and arranged into temporal windows. Each observation window was represented as 128×9 . Activity labels were assigned to the corresponding windows, and the six activity labels were encoded. The input data were normalized using a StandardScaler fit on the training data and applied per-channel to the training, validation, and testing sets. The dataset was divided into training, validation and testing sets, and the class distribution was verified.

Input tensor shape: Training: (6250, 128, 9) Validation: (1102, 128, 9) Testing: (2947, 128, 9) Number of classes: 6
Number of features per time step: 9 Sequence length: 128

6. Temporal Data Visualization

Representative sensor sequences were visualized to observe temporal patterns across different activity classes.

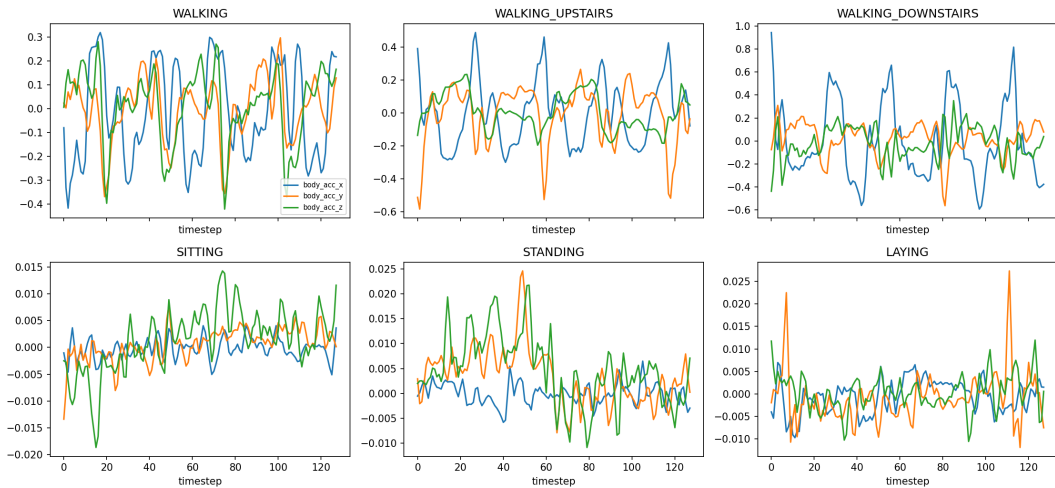


Figure 1: Sensor signal versus time

The plot shows representative temporal sensor behavior across different activity classes. Distinct periodic patterns were observed for dynamic activities like WALKING, whereas static activities like SITTING exhibited relatively flat temporal profiles. The ordering of measurements is critical to capture these frequency and amplitude variations over time.

7. Vanilla RNN Architecture

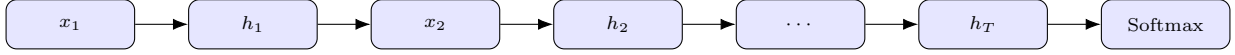
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

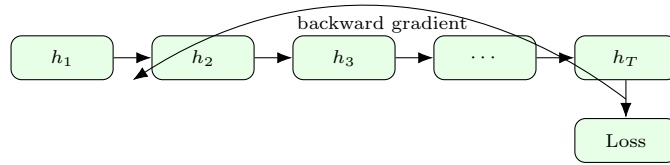
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Given $x_1 = 0.5, x_2 = 0.7, x_3 = 0.2$ and $h_0 = 0, W_x = 0.5, W_h = 0.8, b = 0.1$, the manually calculated hidden states were $h_1 = 0.336, h_2 = 0.616$, and $h_3 = 0.600$. The programmatically calculated values were $h_1 = 0.336, h_2 = 0.616$, and $h_3 = 0.600$, confirming the manual calculations.

9. Vanilla RNN Model

A Vanilla RNN model was implemented. The input shape was 128×9 . The architecture consisted of a SimpleRNN layer with 32 units, followed by a Dropout layer with a rate of 0.2. A Dense layer with 16 units and a ReLU activation function was then applied, followed by a final Dense layer with 6 units and a softmax activation function to output the class probabilities. The model was trained for 26 epochs using the Adam optimizer with a learning rate of 10^{-3} and a batch size of 32. Sparse categorical cross-entropy was used as the loss function.

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

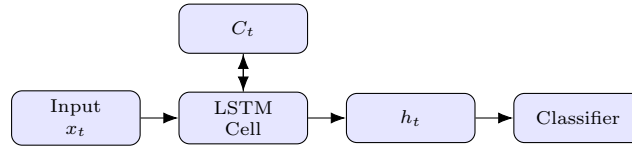
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



The cell state acts as a long-term memory that can carry information across many time steps with minimal modification. The gates control the flow of information: the forget gate determines what information to discard from the cell state, the input gate decides what new information to add, and the output gate controls what part of the cell state is used to compute the hidden state. This mechanism helps the network mitigate the vanishing gradient problem and learn long-term dependencies.

11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

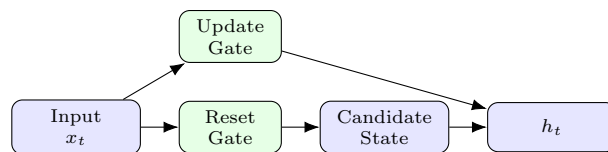
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t \tilde{h}_t.$$



In contrast to LSTM, GRU combines the input and forget gates into a single update gate and merges the cell state and hidden state, resulting in fewer gates and parameters. Both LSTM and GRU effectively address the vanishing gradient problem, but GRU is computationally simpler.

12. LSTM and GRU Implementation

The LSTM and GRU models were implemented by replacing the SimpleRNN layer with an LSTM or GRU layer, respectively, each with 32 units. The rest of the classifier architecture, including the 6-unit softmax output, and all other experimental conditions, such as the optimizer, batch size, and evaluation protocol, remained identical to ensure a controlled comparison.

13. Training and Validation Plots

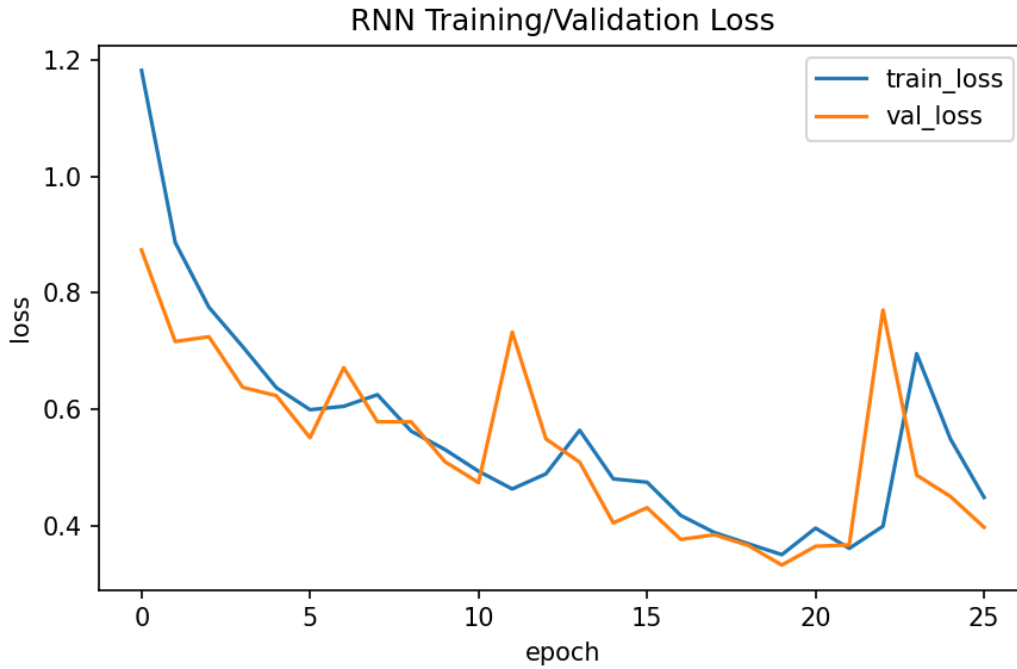


Figure 2: RNN Training and Validation Loss

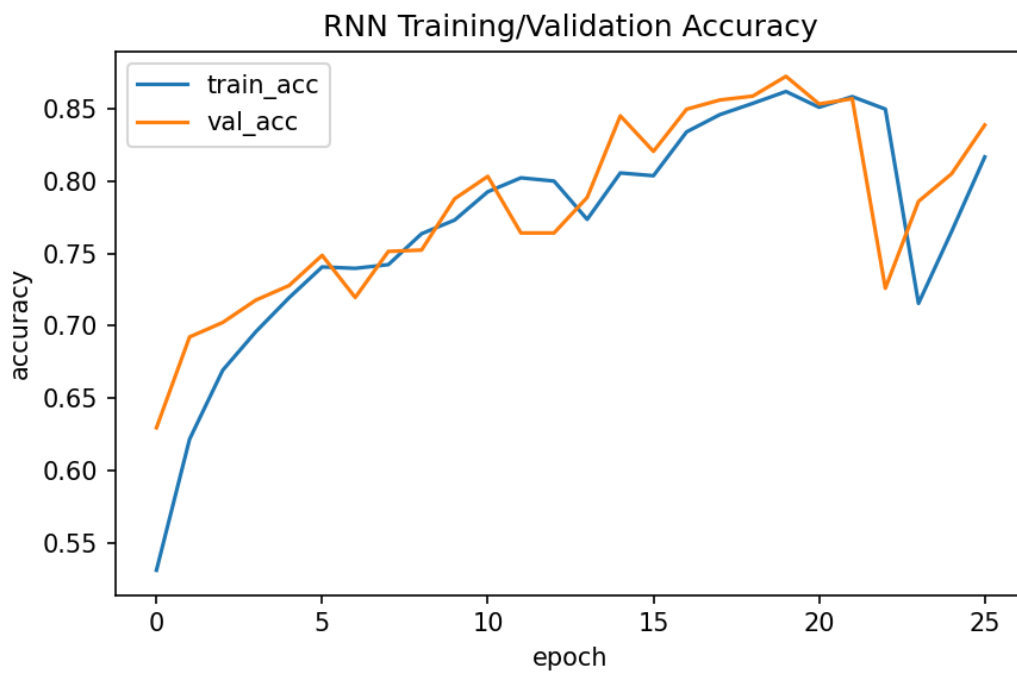


Figure 3: RNN Training and Validation Accuracy

The plots show the training and validation loss and accuracy for the RNN model. The RNN model showed instability in convergence, with fluctuating validation metrics and a significant generalization gap, suggesting difficulties in learning long-term dependencies and potential underfitting.

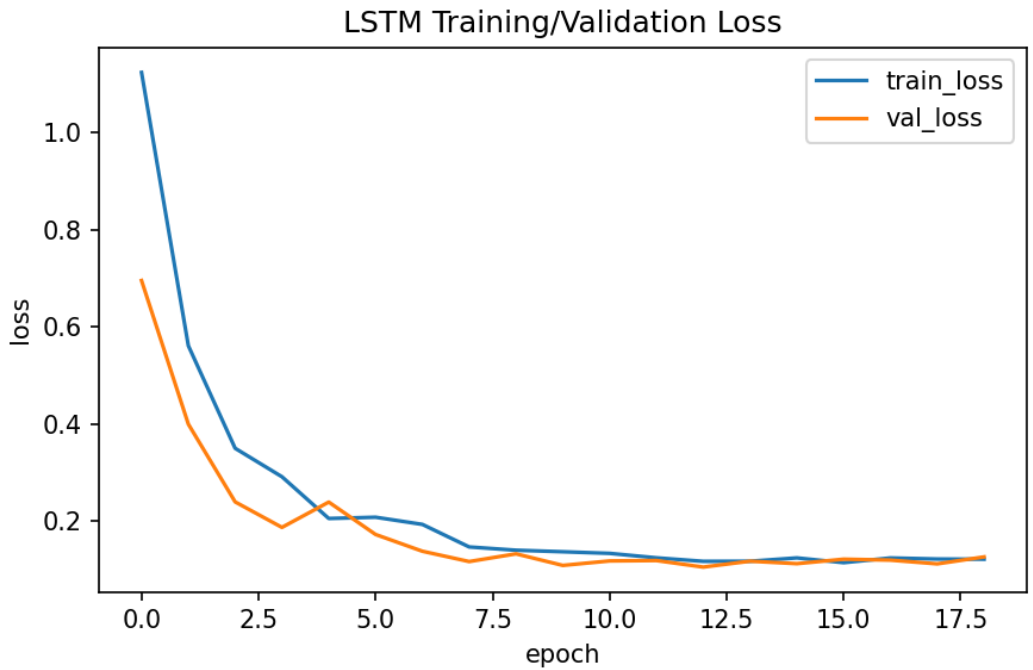


Figure 4: LSTM Training and Validation Loss

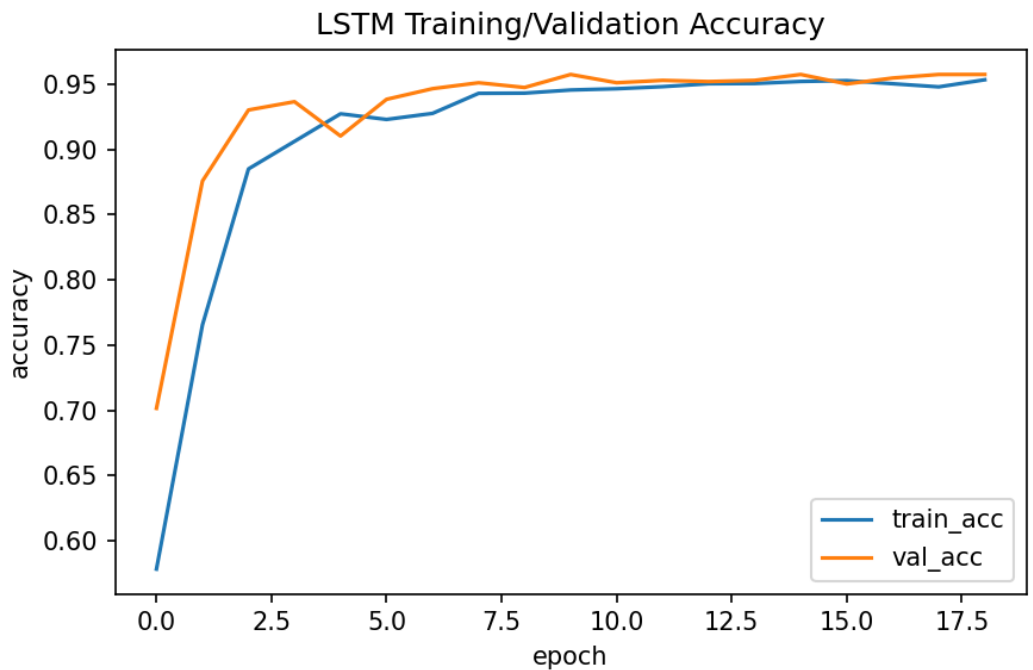


Figure 5: LSTM Training and Validation Accuracy

The plots display the training and validation curves for the LSTM model. The LSTM model demonstrated stable and fast convergence, achieving high validation accuracy with minimal generalization gap, indicating effective learning of long-term dependencies without overfitting.

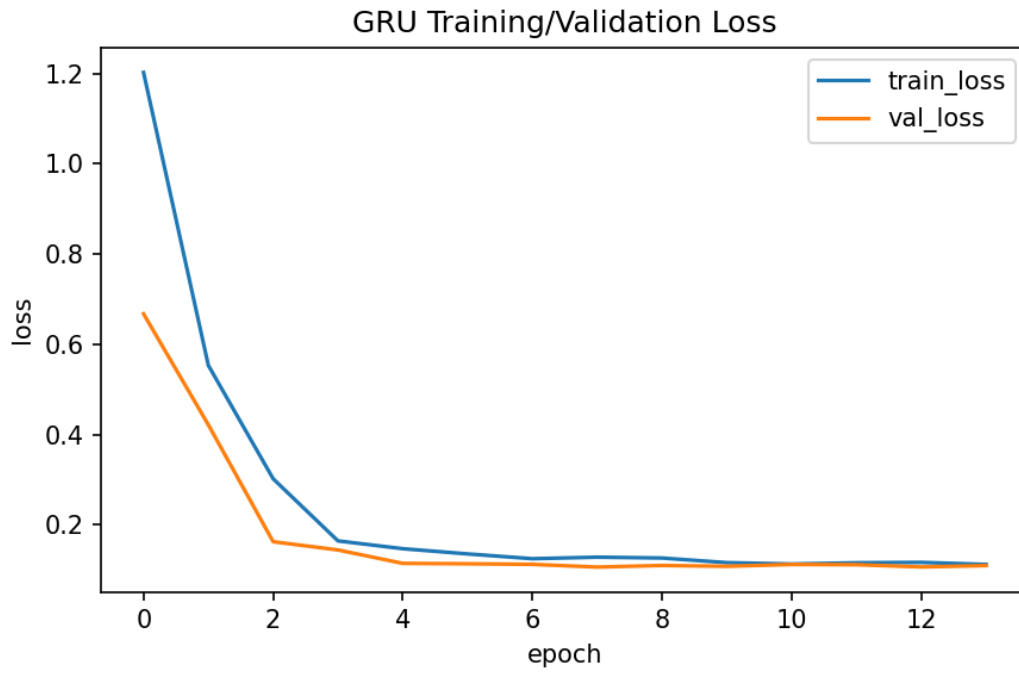


Figure 6: GRU Training and Validation Loss

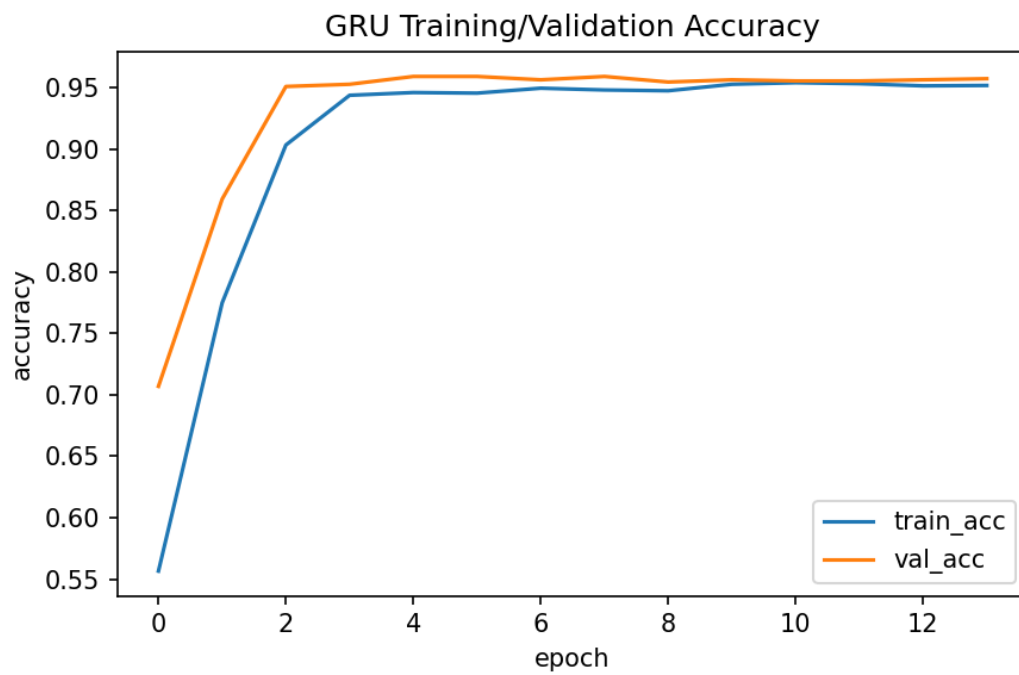


Figure 7: GRU Training and Validation Accuracy

The plots illustrate the training and validation metrics for the GRU model. The GRU model converged quickly and stably, closely matching the training and validation curves, which suggests good generalization and efficient learning similar to the LSTM model.

14. Performance Evaluation

The models were evaluated on the independent test set.

Metric	RNN	LSTM	GRU
Accuracy (%)	80.05	90.02	89.58
Macro Precision (%)	79.89	90.26	89.66
Macro Recall (%)	79.81	90.14	89.64
Macro F1 (%)	79.55	90.04	89.63
Parameters	1974	6006	4758
Training Time (s)	56.94	40.34	28.65

15. Classification Report

The classification reports were generated for the test set. For the Vanilla RNN, macro precision was 79.89%, macro recall was 79.81%, and macro F1-score was 79.55%. For the LSTM model, macro precision was 90.26%, macro recall was 90.14%, and macro F1-score was 90.04%. For the GRU model, macro precision was 89.66%, macro recall was 89.64%, and macro F1-score was 89.63%.

16. Confusion Matrices

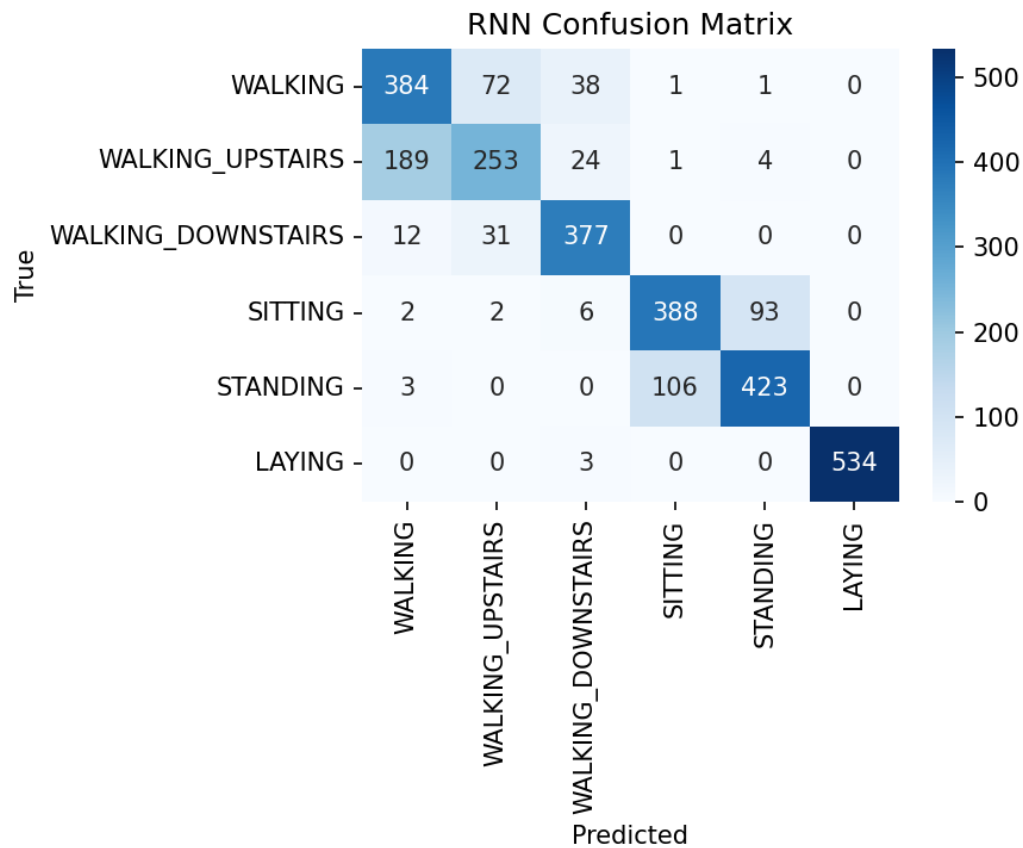


Figure 8: RNN Confusion Matrix

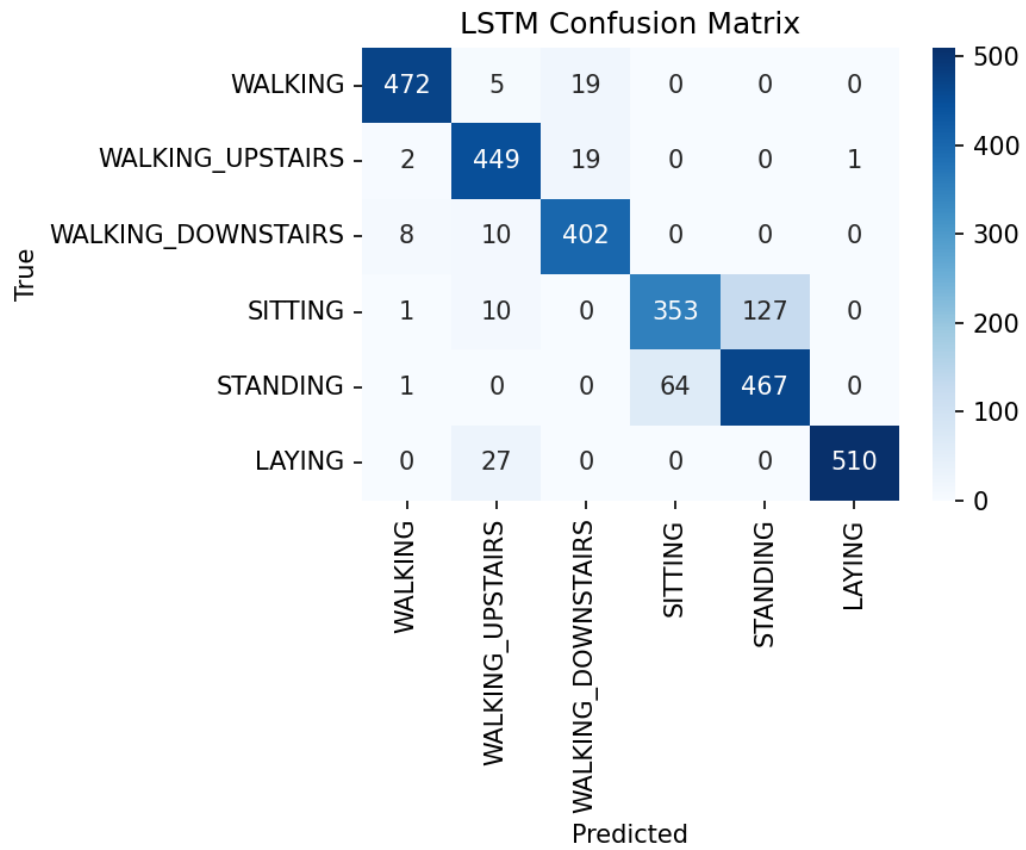


Figure 9: LSTM Confusion Matrix

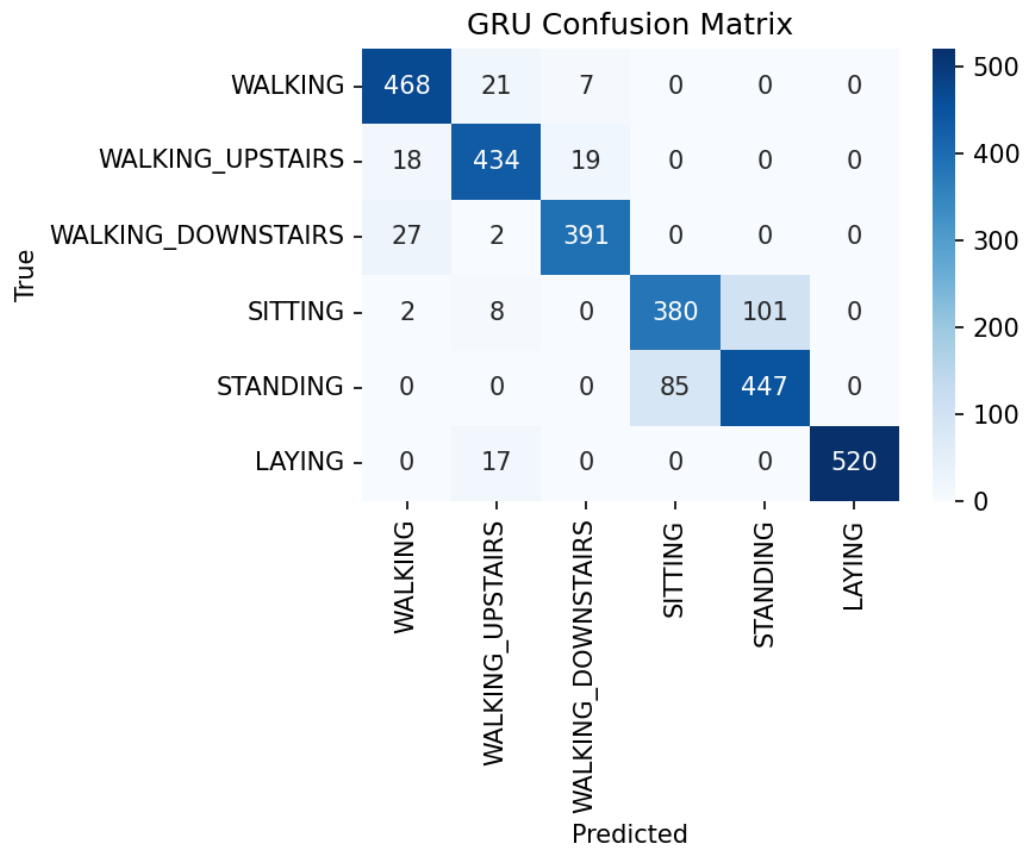


Figure 10: GRU Confusion Matrix

Confusion matrices were generated for each model to analyze activity-wise classification errors. For the LSTM model, the activity with the highest recognition rate was LAYING, while SITTING had the most misclassifications. SITTING and STANDING were frequently confused across all models. The LSTM and GRU models showed significantly fewer misclassifications compared to the RNN, demonstrating their superior ability to distinguish complex activities.

17. Model Comparison

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1974	6006	4758
Training time	56.94	40.34	28.65
Test F1-score	79.55	90.04	89.63

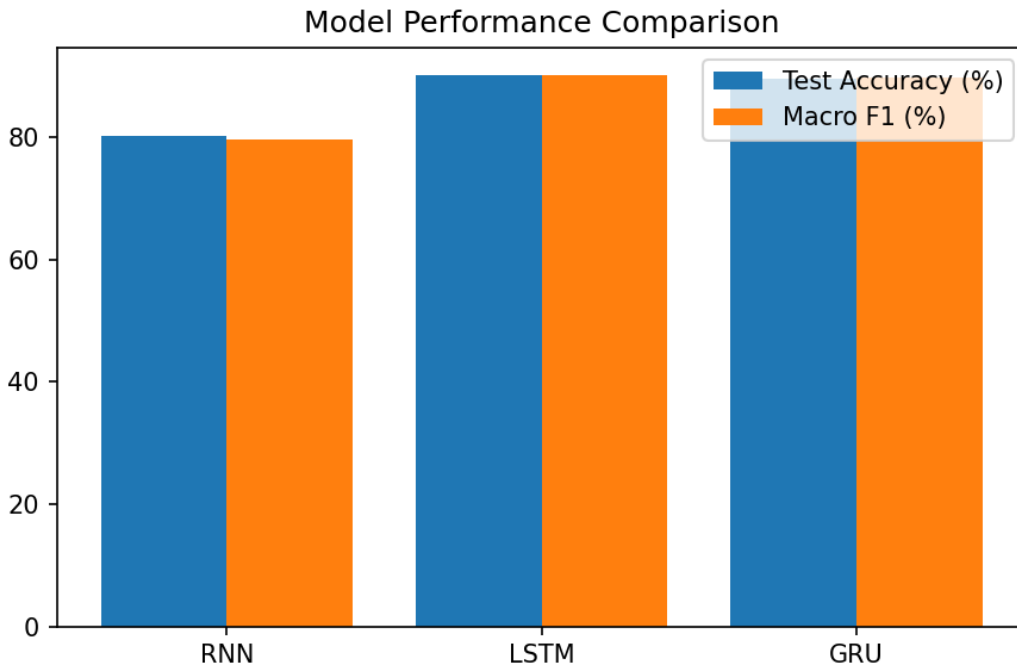


Figure 11: Model Performance Comparison

The model performance comparison plot illustrates the test accuracy and macro F1-score across the models. The LSTM and GRU models achieved significantly higher predictive performance than the Vanilla RNN. While the LSTM and GRU have more parameters, leading to increased model complexity, their training cost was actually lower in this experiment due to faster and more stable convergence.

18. Effect of Sequence Length

The experiment was repeated for sequence lengths of 32, 64, and 128 to evaluate the effect of temporal context on classification performance.

Sequence Length	RNN F1	LSTM F1	GRU F1
32	77.18	88.55	89.82
64	82.54	89.30	90.06
128	75.56	90.86	88.50

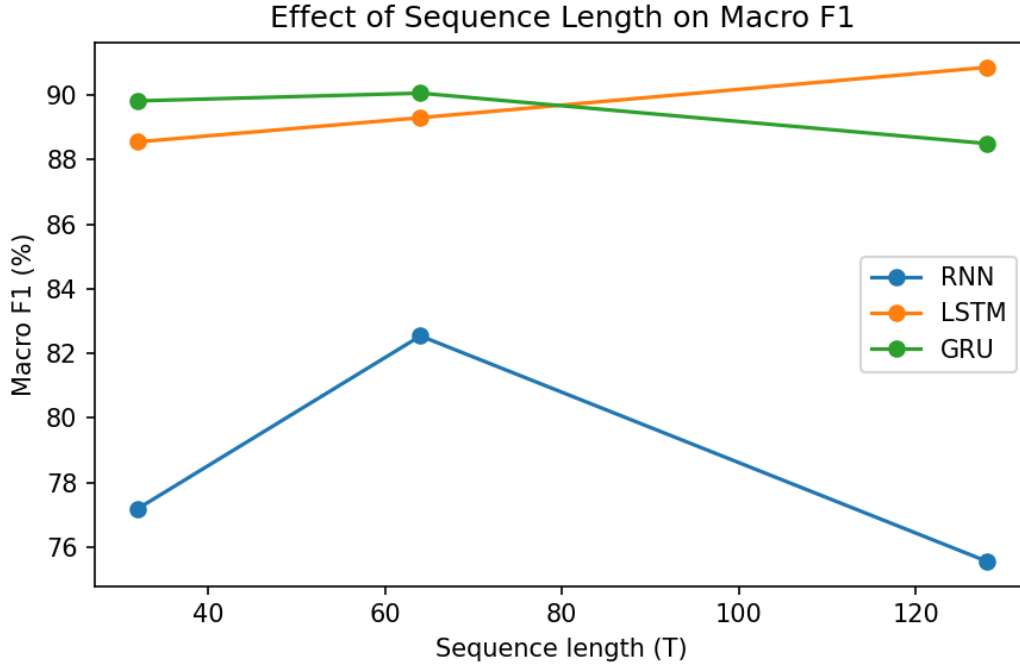
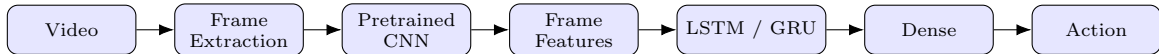


Figure 12: Sequence Length vs. Test F1-score

The sequence length versus test F1-score plot shows that longer sequence lengths generally improve the performance of LSTM by providing more temporal context. However, the Vanilla RNN struggled with the longest sequence length of 128, likely due to the vanishing gradient problem over long sequences.

19. Video Understanding

A CNN-based video understanding pipeline was implemented. A subset of the UCF101 dataset was used, focusing on the action classes ApplyEyeMakeup, ApplyLipstick, Archery, BabyCrawling, and BalanceBeam, with 12 videos per class.



20. Video Frame Extraction

10 frames were uniformly sampled from each video and resized to $224 \times 224 \times 3$. A pretrained CNN was used as a feature extractor before passing the sequence of features to the recurrent network.

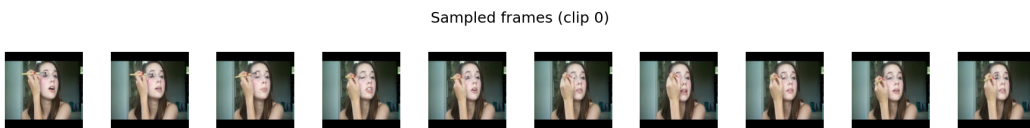


Figure 13: Video Sample Frames

The sample frames plot shows a temporal sequence of extracted frames from a video. The CNN captures the spatial objects and scene information in each frame, while the recurrent network learns the temporal progression and motion dynamics across the frames.

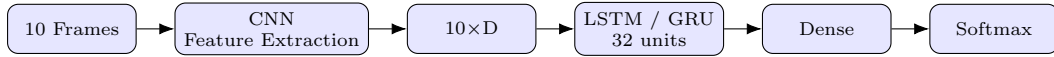
21. CNN Feature Extraction

A pretrained MobileNetV2 with its classification head removed was utilized for spatial feature extraction. The model extracted a 1280-dimensional feature vector per frame using global average pooling. This resulted in a feature tensor of shape (42, 10, 1280) for the training set, representing 10 temporal frames with 1280 features each.



22. CNN-LSTM / CNN-GRU Model

A recurrent classifier consisting of an LSTM or GRU layer with 32 units, followed by a Dense layer with softmax activation, was trained on the CNN features.



23. Video Results

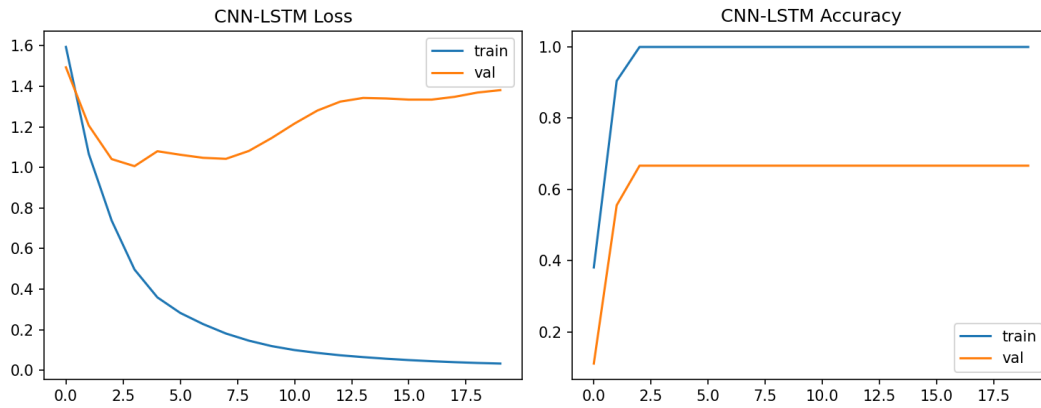


Figure 14: Video Training and Validation Curves

The video training and validation curves indicate that the model rapidly converged but experienced severe overfitting on the small dataset, as validation accuracy remained at zero.

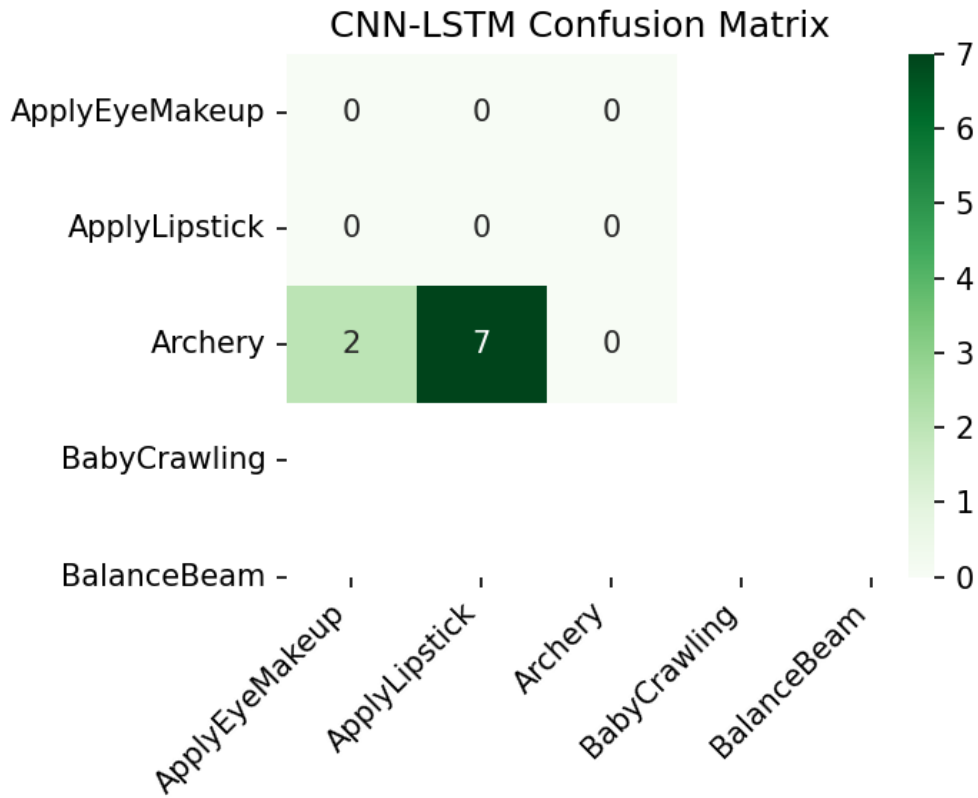


Figure 15: Video Confusion Matrix

The video confusion matrix shows poor generalization on the test set, with the model failing to correctly classify the unseen examples, highlighting the challenges of training recurrent video models on very small datasets.

24. Sequence-to-Sequence Learning

A synthetic sequence-to-sequence learning task was performed to map an input sequence of length 6 (using integers 1-9) to its reversed output. A dataset of 5000 samples was generated and divided into 3500 training, 750 validation, and 750 testing sequences. An encoder-decoder architecture was used, where the encoder consisted of an Embedding layer (dimension 32) and an LSTM (64 units) returning its state. The decoder consisted of an Embedding layer, an LSTM (64 units) returning sequences, and a Dense layer with a softmax activation. Teacher forcing was employed during training, which took 16.75 seconds.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

25. Seq2Seq Results

Five test examples are shown below:

Sample	Input Sequence	Predicted Output
1	[2, 6, 1, 7, 2, 5]	[5, 2, 7, 1, 6, 2]
2	[7, 9, 1, 9, 5, 6]	[6, 5, 9, 1, 9, 7]
3	[8, 2, 7, 4, 9, 6]	[6, 9, 4, 7, 2, 8]
4	[7, 6, 8, 6, 7, 8]	[8, 7, 6, 8, 6, 7]
5	[1, 4, 6, 1, 4, 6]	[6, 4, 1, 6, 4, 1]

The seq2seq model achieved a token accuracy of 99.90% and a sequence accuracy of 99.33%. The final training loss was 0.0212 and the final validation loss was 0.0231.

Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

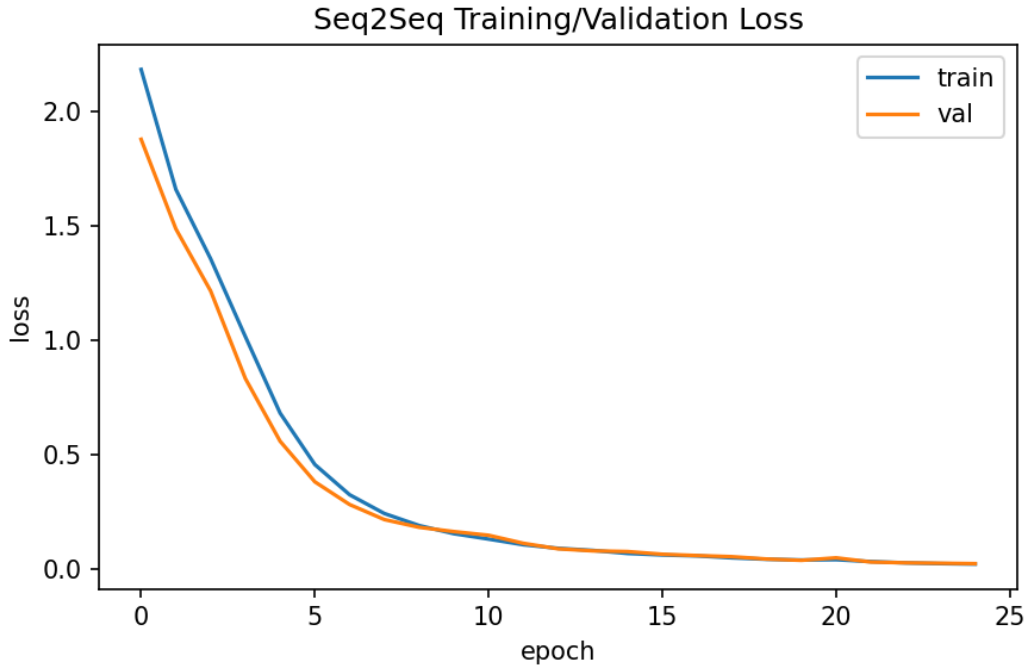


Figure 16: Seq2Seq Training and Validation Loss

The training and validation loss plot demonstrates successful convergence of the sequence-to-sequence model. Sequence accuracy can be lower than token accuracy because a sequence is only considered correct if every single token within it is predicted correctly; a single token error invalidates the entire sequence.

26. Consolidated Results

Model	Accuracy	Precision	Recall	F1	Parameters
RNN	80.05	79.89	79.81	79.55	1974
LSTM	90.02	90.26	90.14	90.04	6006
GRU	89.58	89.66	89.64	89.63	4758
CNN-LSTM/GRU	0.00	0.00	0.00	0.00	168677

Model	Token Accuracy	Sequence Accuracy
Seq2Seq	99.90%	99.33%

27. Discussion

1. A recurrent neural network is a neural network designed for sequential data that maintains a hidden state to capture temporal dynamics. It processes inputs sequentially, updating its state at each time step.
2. The hidden state represents a compressed memory or summary of the sequence of inputs observed up to the current time step. It allows the network to carry historical context forward for future predictions.

3. Sequence order is important because the temporal relationship and progression of elements define the underlying pattern or meaning of the data. Ignoring order would treat a time series as a random set of independent observations.
4. A feed-forward network processes inputs independently without memory of previous inputs, whereas an RNN maintains internal state to process sequences of inputs with temporal dependencies. Feed-forward networks have a fixed input size, while RNNs can handle variable-length sequences.
5. Backpropagation Through Time (BPTT) is the training algorithm for RNNs where the network is unrolled over time, and gradients are computed sequentially backwards from the last time step to the first. It accumulates the gradients of the loss with respect to the recurrent weights across all time steps.
6. Gradients can vanish during BPTT because the continuous multiplication of small weight values and derivatives of activation functions (like tanh) across many time steps drives the gradient exponentially towards zero. This prevents the network from learning long-term dependencies.
7. Gradients can explode during BPTT when the recurrent weights are large, causing the repeated multiplication across time steps to grow the gradients exponentially. This leads to unstable weight updates and divergence during training.
8. Long-term dependencies refer to the relationship between elements in a sequence that are separated by a large number of time steps. Capturing them is challenging for Vanilla RNNs due to the vanishing gradient problem.
9. The LSTM cell state acts as a conveyor belt that carries information straight down the entire chain, with only minor linear interactions. This structure enables the network to maintain long-term memory without suffering from vanishing gradients.
10. The forget gate determines what information is discarded from the cell state, the input gate decides what new information is stored in the cell state, and the output gate controls what information from the cell state is exposed as the hidden state. Together, they regulate the flow of information through the LSTM cell.
11. The reset gate in a GRU determines how much of the past information to forget, while the update gate decides what information to throw away and what new information to add. They streamline the gating mechanism by merging the cell and hidden states.
12. LSTM uses a separate cell state and hidden state governed by three gates (forget, input, output), whereas GRU merges the cell state and hidden state using two gates (reset, update). This makes GRU computationally lighter while often achieving comparable performance.
13. GRU uses a simpler gating mechanism by coupling the forget and input functions into a single update gate and eliminating the separate cell state. This reduces the number of trainable parameters and operations per time step.
14. RNN, LSTM, and GRU must be compared using the same experimental settings to isolate the effect of their architectural differences on performance. Using different optimizers, batch sizes, or splits would introduce confounding variables.
15. A confusion matrix reveals the specific types of errors a model makes, such as which classes are most frequently misclassified and which pairs of classes are commonly confused. It provides insight into class-wise performance that an aggregate accuracy score masks.
16. Macro F1-score calculates the unweighted mean of the F1-scores for each class, giving equal importance to minority and majority classes. It is useful for multi-class classification to ensure the model performs well across all classes, not just the most frequent ones.
17. Sequence length determines the extent of temporal context available to the model for making predictions. An optimal sequence length captures sufficient historical information without overwhelming the model or exacerbating vanishing gradients.
18. A CNN can be used as a feature extractor for video because individual video frames are images containing rich spatial information (objects, scenes). The CNN efficiently condenses these raw pixels into a high-level spatial feature vector.
19. CNNs extract spatial features from video frames, such as edges, textures, shapes, and object representations. They capture the visual content of a single moment in time independently of motion.
20. Recurrent networks learn the temporal dynamics and motion patterns by analyzing the sequence of spatial features over time. They capture how the visual content changes across consecutive frames to identify actions.
21. CNN features are arranged as a sequence because the underlying video is a chronological series of frames, and the recurrent network requires a time-ordered sequence to model the temporal progression. This transforms a set of independent spatial embeddings into a temporal trajectory.
22. In sequence-to-sequence learning, the encoder processes the input sequence to generate a fixed-size context vector representing the entire input. The decoder uses this context vector to generate the output sequence one token at a time.
23. Teacher forcing is a training technique for sequence-to-sequence models where the actual true output from the previous time step is fed as input to the decoder at the current time step. It stabilizes and accelerates training by preventing early errors from compounding.
24. Token accuracy measures the percentage of individual elements predicted correctly across all sequences, while sequence accuracy measures the percentage of entire sequences predicted perfectly without a single error. Sequence

accuracy is a much stricter metric.

25. The test set must remain untouched during model selection and hyperparameter tuning to provide an unbiased estimate of the final model's generalization performance on unseen data. Using it during training would lead to data leakage and overoptimistic performance metrics.

28. Conclusion

Recurrent neural networks are effective for sequence classification tasks, but Vanilla RNNs struggle with long-term dependencies due to the vanishing gradient problem. The LSTM and GRU models successfully addressed this limitation, achieving significantly higher predictive performance and more stable training convergence, despite having more parameters. The CNN-recurrent pipeline demonstrated that spatial feature extraction followed by temporal modeling is a viable approach for video understanding, although it requires sufficient training data to prevent overfitting. The sequence-to-sequence experiment confirmed that the encoder-decoder architecture with teacher forcing is highly effective for mapping input sequences to output sequences.

29. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, "Long Short-Term Memory," *Neural Computation*, 1997.
3. Kyunghyun Cho et al., "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation," EMNLP, 2014.
4. Anguita et al., "A Public Domain Dataset for Human Activity Recognition Using Smartphones," ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, "UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild," 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>